

Junta de Andalucía
Consejería de Educación y Deporte

Diseño de Interfaces Web
2º DAW
Unidad Didáctica 5

MODELOS DE DISEÑO. REVISIÓN HTML5. TABLAS. MARCOS.

Francisco Jesús Gómez Romero, 2026/27

(cc) BY-NC-SA

Objetivos



- Utilizar **gestores de contenido**

- **Contextualizar** el diseño de interfaces web
- Conocer los **elementos principales** de una **Interfaz Web**
- Analizar cómo influye la parte **visual** de una web en su cometido
- **Planificar** el diseño de interfaces web.
- Crear guías **guías de estilo**
- Aplicar **patrones de diseño**

Índice de contenidos

1. Modelos de diseño
   1.1. Repaso lenguaje de marcas
   1.2. Diseño con tablas vs diseño por capas
   1.3. Iframes
2. CMS

RA y CE

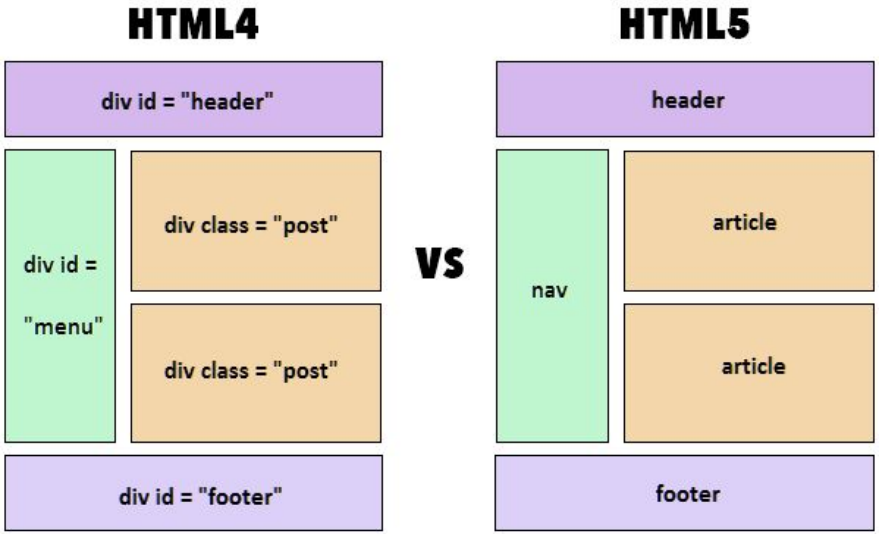
RA1. Planifica la creación de una interfaz web valorando y aplicando especificaciones de diseño.
a) Se ha reconocido la importancia de la comunicación visual y sus principios básicos.
b) Se han analizado y seleccionado los colores y tipografías adecuados para su visualización en pantalla.
c) Se han analizado alternativas para la presentación de la información en documentos Web.
d) Se ha valorado la importancia de definir y aplicar la guía de estilo en el desarrollo de una aplicación Web.
e) Se han utilizado y valorado distintas aplicaciones para el diseño de documentos Web.
f) Se han utilizado marcos, tablas y capas para presentar la información de manera ordenada.
g) Se han creado y utilizado plantillas de diseño.

Repaso Lenguajes de Marcas

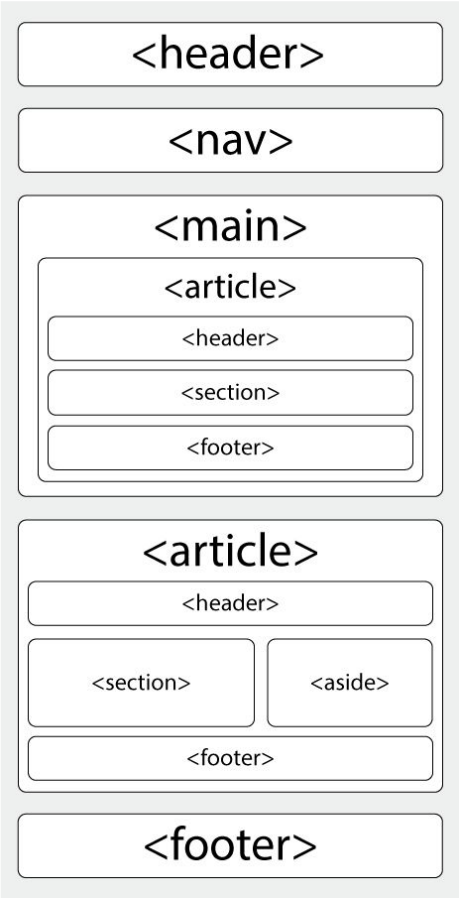
Un lenguaje de marcas es la forma de codificar un documento que, junto con el texto, incorpora etiquetas o marcas que contienen información adicional acerca de la estructura del texto o su presentación.

Tipos de Marcado:

- Presentación
- Procedimiento
- Semántico



Recordemos un poco de HTML





HTML Quiz

Algunos elementos que debemos conocer ya:

- iframe
- form
- Img
- …

Ejercicio 1

Crea un archivo llamado index.html donde escribirás el código HTML que contenga:

Cabecera (Header) con el título de la página. El texto debe estar alineado en el centro dentro del encabezado principal (`<h1>`).
Navegación (Nav) con un menú simple de 3 enlaces (pueden ser falsos, no necesitan ir a otras páginas). El menú de navegación debe usar una **lista no ordenada** (`<ul>`).
Cuerpo (Main) con:

- Un **encabezado** (título principal) para el contenido.
- Un **párrafo** describiendo tu página o un tema de tu elección.
- Una **imagen** relacionada con el contenido.

Pie de página (Footer) con un texto simple (por ejemplo, los derechos de autor o tu nombre).

Declaración del tipo de documento `<!DOCTYPE html>`.
Etiquetas de apertura y cierre de `<html>`, `<head>`, `<body>`.
Un elemento `<meta charset="UTF-8">` para definir el juego de caracteres.

Diseño Basado en Tablas (Obsoleto)

Este método se utilizaba antes de la estandarización de CSS y la proliferación de dispositivos móviles. Se trata de usar las etiquetas HTML <table>, <tr> (fila) y <td> (celda) no para datos tabulares, sino para **estructurar la *totalidad* del diseño** (cabecera, barra lateral, pie de página, etc.).

Uso Incorrecto de la Semántica	Se usa la tabla para la **disposición visual**, en lugar de para su propósito original: mostrar **datos tabulares**.
Rigidez y No Responsivo	El diseño es **rígido** y se basa en anchos fijos de píxeles, lo que lo hace totalmente **no apto para dispositivos móviles** (no es *responsive*).
Mantenimiento Difícil	Los estilos y la estructura están mezclados en el HTML, haciendo que cualquier cambio de diseño requiera modificar *todo* el código de la tabla.
Accesibilidad y SEO	Dificulta la lectura del contenido a lectores de pantalla (para accesibilidad) y a los motores de búsqueda (para SEO), ya que el orden del código no refleja la jerarquía real del contenido.

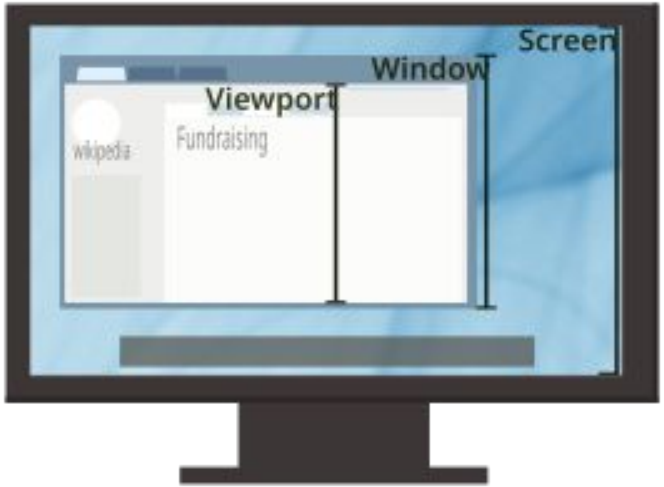
Diseño Basado en Capas (Actual)

Este es el enfoque moderno, basado en el principio de la **Separación de Intereses**. Se utiliza HTML para la **estructura semántica** y CSS para la **presentación** (disposición y estilos).

Característica	Descripción
Separación de Responsabilidades	**HTML** define la **estructura** y significado (<header>, <main>, <footer>). **CSS** define la **presentación** y el diseño (display: flex;, grid-template-areas;).
Diseño Flexible (Responsivo)	Permite usar unidades relativas (%, em, vw) y herramientas como **Flexbox** y **CSS Grid** para crear diseños fluidos y adaptables a *cualquier* tamaño de pantalla.
Mantenimiento Sencillo	Puedes modificar completamente el diseño de la web (moviendo elementos o cambiando colores) editando solo la hoja de estilos CSS, sin tocar el HTML.
Accesibilidad y SEO	El código es limpio y semántico. Los motores de búsqueda y los lectores de pantalla interpretan la jerarquía del contenido de forma correcta.

Modelos diseño web: conceptos

- **Viewport:** área visible del navegador donde se muestra el contenido de una página web
- **Layout**: determina la **disposición de los elementos** dentro del viewport.
- **Pixel**: cada punto que se muestra en la pantalla
- Unidad **absoluta**: no se escala
- Unidad **relativa**: sí se escala



Modelos diseño web: Tipos

Según se adapta el diseño a la pantalla, existen distintos modelos:

- Fijo
- Fluido
- Elástico
- Adaptativo
- Responsivo



Ancho fijo

- Medidas absolutas: px `width="320px"`
- No cambia el ancho → Scroll
- El ancho más usado es 1920 px (full HD)



- Ventaja: control sobre el tamaño de los elementos: texto, imágenes…

- Desventaja:
  - Aparición del scroll cuando se supera el ancho de la pantalla
  - Espacios vacíos cuando se reduce

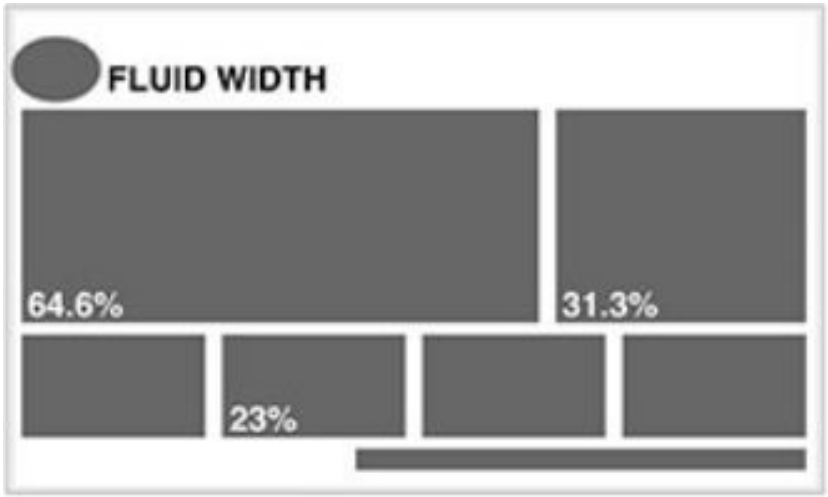
Ancho variable (líquido o fluido)

- Medidas relativas: % `width="33%"`
- El ancho se adapta a la pantalla

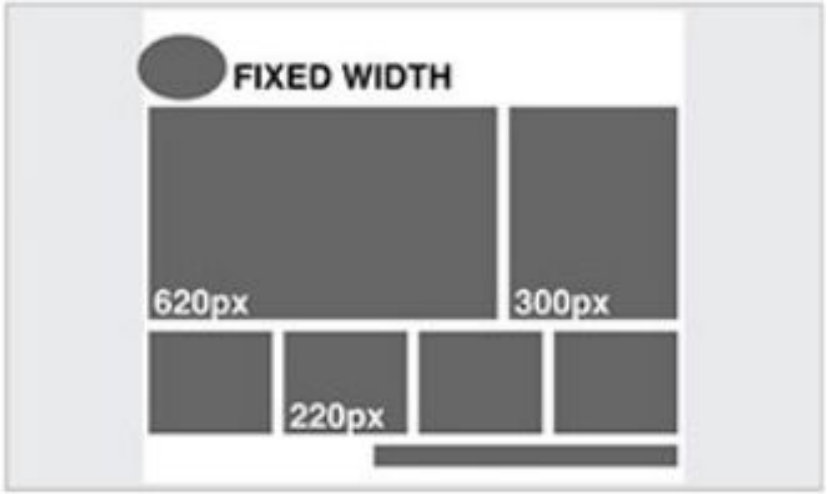


- Ventaja:
  - Mejor adaptación al espacio de la pantalla
- Desventaja:
  - en pantallas muy grandes o muy pequeñas, los contenidos se estiran demasiado o se contraen demasiado

Fijo vs Variable



Fluid Width: Content spanning the entire page



Fixed Width: Content remains centered

Diseño Híbrido

- Combina áreas con diseño fijo y áreas con diseño fluido

```
<table max-width="960" border="0" cellpadding="10" cellspacing="0">
    <tr>
        <td colspan="3" bgcolor="#dddddd" align="center">
            <h1>Mi Página</h1>
            <nav>
                <a href="#">Inicio</a> |
                <a href="#">Servicios</a> |
                <a href="#">Contacto</a>
            </nav>
        </td>
    </tr>
    <!-- Cuerpo principal con tres columnas -->
    <tr>
        <td width="33%" valign="top" bgcolor="#eeeeee">
            <h2>Columna 1</h2>
            <p>Contenido de la columna 1.</p>
        </td>
```

Diseño Elástico

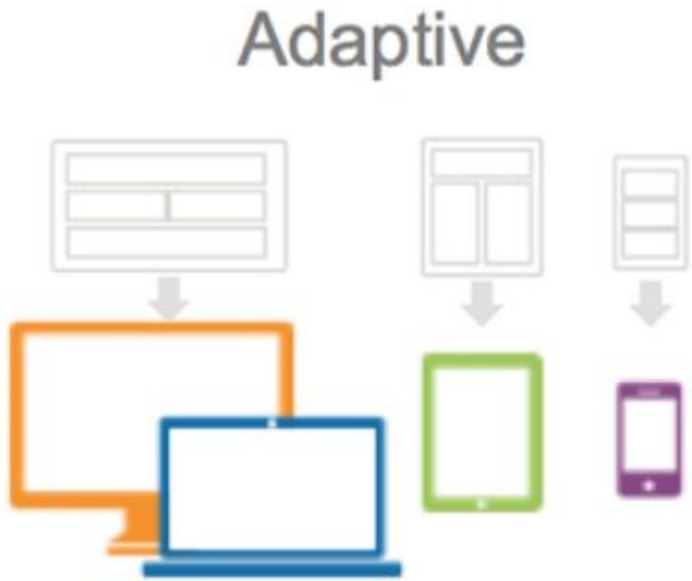
- Igual que el variable, pero en lugar de ser relativo al tamaño del contenedor es **relativo al tamaño del texto** del contenedor.

- Se consigue usando el tamaño relativo a la fuente: em y rem

- Ventajas:
  - Cambiar el tamaño de la fuente también afecta al tamaño de los contenedores → útil para accesibilidad

Diseño Adaptativo (AWD)

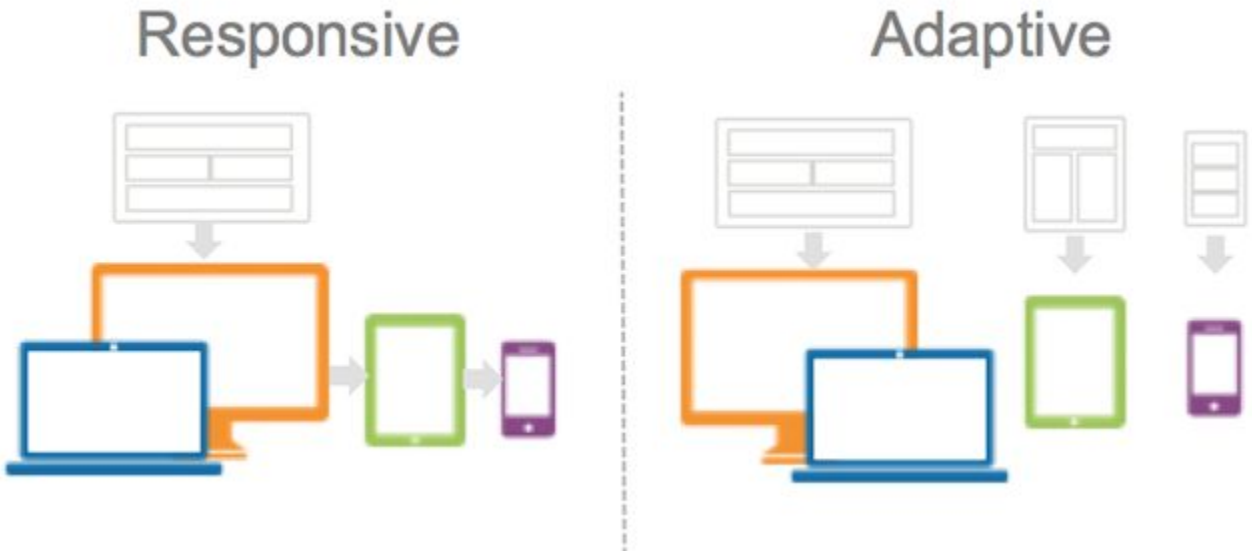
- Puntos de salto (breakpoints): Diseñar la web en varias plantillas distintas según el dispositivo.
- Cuando se alcanza un breakpoint se carga la plantilla correspondiente
- Los anchos más comunes son 320px, 480px, 760px, 960px, etc.



Ventajas	Desventajas
Mayor control para personalizar la experiencia en cada tamaño fijo	Mayor esfuerzo inicial y de mantenimiento

Diseño Responsivo o Sensible (RWD)

- El diseño y el contenido se adaptan a cada pantalla de manera **fluida** y no a saltos
- Los contenidos se ordenan en bloques que se reorganizan según las características de la pantalla, permitiendo usar **una sola plantilla** para todos los dispositivos.
- Especialmente útil para móviles



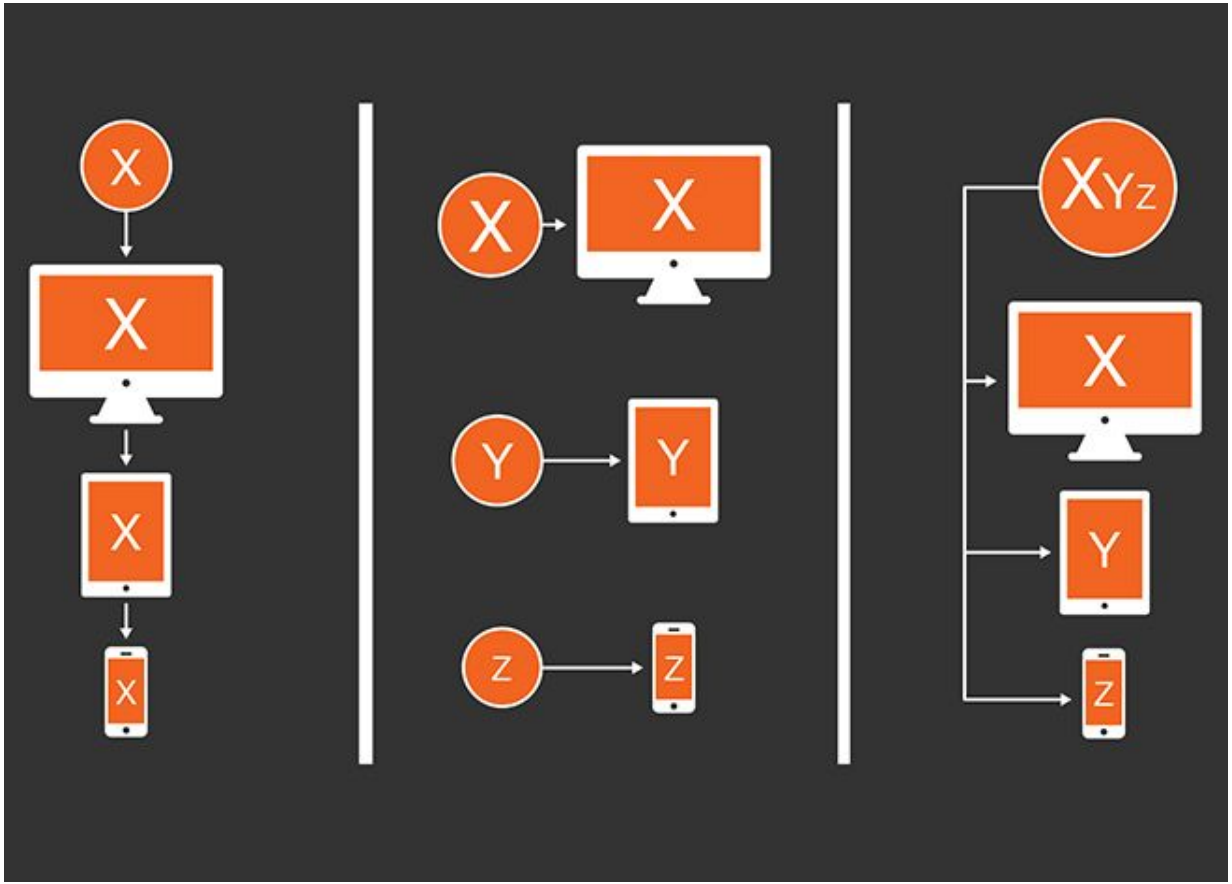
Diseño Responsivo o Sensible (RWD)

- Se basa en una **única plantilla** que utiliza unidades de medida **relativas** (`%`, `em`, `vw`) y **Media Queries** de CSS.

Unidad	Relativa a...	Uso Recomendado
`%`	El ancho o alto del **elemento contenedor (padre).**	Dimensionar contenedores de *layout* internos (ej: una columna de un *layout* fluido puede ser `width: 70%;`).
`vw` (*Viewport Width*)	El **1% del ancho** del *viewport* (ventana visible).	Dimensionar elementos basándose en el ancho total de la pantalla, como tamaños de fuente que deben escalar con el navegador (*fluid typography*), o anchos de contenedores.
`vh` (*Viewport Height*)	El **1% de la altura** del *viewport* (ventana visible).	Elementos que deben ocupar toda o una parte específica de la altura de la pantalla (ej: un *hero section* de `height: 100vh;`).

Ventajas	Desventajas
Más flexibilidad y menor esfuerzo inicial y de mantenimiento	Menos control sobre los elementos

Ejercicio 2



Ejercicio 3

Busca un ejemplos en la web para cada uno de los modelos:

1. Fijo
2. Líquido
3. Adaptativo
4. Responsivo

Si no los encuentras genera con IA una página web que sirva de ejemplo para cada uno.

Ejercicio 4: ancho fijo.

Crea una página web utilizando solo HTML en la que se aplique un diseño de ancho fijo.

El objetivo es que los elementos de la página se muestren organizados de forma visual y manteniendo siempre un ancho fijo, sin importar el tamaño de la ventana del navegador.

La página debe tener un ancho fijo de 800 píxeles

¿Serías capaz de centrarla?

Ejercicio 5: ancho variable

Implementar la página anterior con ancho variable en un nuevo fichero HTML y comparar el resultado.